

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

def insert_text_in_tex(file_path: str, insertion_keyword: str, text) -> bool:
    """
    Inserts `text` (string or list of strings) into a .tex file,
    replacing everything between the markers:

    % BEGIN <insertion_keyword>
    (old lines here)
    % END <insertion_keyword>

    Returns True on success, False if markers aren't found.
    """
    # Build the exact marker lines
    start_tag = f"% BEGIN {insertion_keyword}"
    end_tag = f"% END {insertion_keyword}"

    # Normalize `text` to a list of lines (each ending in `\n`)
    if isinstance(text, str):
        lines_to_insert = [text.rstrip("\n") + "\n"]
    elif isinstance(text, list):
        lines_to_insert = [str(t).rstrip("\n") + "\n" for t in text]
    else:
        raise TypeError("`text` must be a str or a list of str")

    # Read the file
    with open(file_path, "r", encoding="utf-8") as f:
        lines = f.readlines()

    # Find the markers
    start_idx = next((i for i, L in enumerate(lines) if L.strip() == start_tag), None)
    end_idx = next((i for i, L in enumerate(lines) if L.strip() == end_tag), None)

    if start_idx is None or end_idx is None or end_idx <= start_idx:
        return False

    # Replace everything *between* the markers by slice assignment
    # (this preserves the lines at start_idx and end_idx)
    lines[start_idx+1 : end_idx] = lines_to_insert

    # Write it back
    with open(file_path, "w", encoding="utf-8") as f:
        f.writelines(lines)

    return True

#Load pollution analysis
#From Table 3- 15. use the following average constituent contributions:
bod_load_g_per_capita = 60 # g/person/day, range 50-120 g/person/day
tss_load_g_per_capita = 60 # g/person/day, range 60-150 g/person/day

Q_domesticpercapita=162 #L/PE/day taken from standards.\n",
bod_concentration= round(bod_load_g_per_capita/Q_domesticpercapita*1000,2)# mg/L\n",
print('BOD average concentration: '+str(bod_concentration)+ 'g/m3')
tss_concentration= round(tss_load_g_per_capita/Q_domesticpercapita*1000,2)# mg/L\n",
print ('TSS average concentration : '+str(tss_concentration)+ 'g/m3')

BOD average concentration: 370.37g/m3
TSS averge concentration : 370.37g/m3

```

✓ Estimation of design flows and pollutant loads.

```

import math
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

```

```
#Estimation of design flows and pollutant loads.
```

```
group_number=10
PE_domestic=340000 #Equivalent population.
Q_domesticpercapita=162 #L/PE/day taken from standards.
PE_commercial=math.ceil(group_number**0.5*9000) #Equivalent population.
Q_commercialpercapita=280 #L/PE/day given in the briefing.
Q_infiltration=11657 #m3/day
```

```
#Current average flow rates
Q_currentavgdomestic=Q_domesticpercapita*PE_domestic/1000
Q_currentavgcommercial=Q_commercialpercapita*PE_commercial/1000
Q_currentavginfiltration=Q_infiltration
```

```
#Current minimum flow rates
peak_minimum_factor=0.3 #Taken from the graph given in the briefing
Q_minimumdomestic=Q_currentavgdomestic*peak_minimum_factor
Q_minimumcommercial=Q_currentavgcommercial*peak_minimum_factor
Q_minimuminfiltration=Q_currentavginfiltration
```

```
#Current maximum flow rates
peak_maximum_factor=1.9 #Taken from the graph given in the briefing
Q_maximumdomestic=Q_currentavgdomestic*peak_maximum_factor
Q_maximumcommercial=Q_currentavgcommercial*peak_maximum_factor
Q_maximuminfiltration=Q_currentavginfiltration
```

```
#Equivalent population projection
growth_percentage=1+group_number**0.015
Life_time=20 #Life time decided to our WWTP. Provisional
```

```
PE_projected_domestic=math.ceil(PE_domestic*(1+growth_percentage/100)**Life_time)
PE_projected_comercial=math.ceil(PE_commercial*(1+growth_percentage/100)**Life_time)
```

```
climate_chage_factor=1.2 #Taken from a projection studies.
```

```
#Projected average flow rates (m3/day)
Q_projected_domestic=Q_domesticpercapita*PE_projected_domestic/1000
Q_projected_comercial=Q_commercialpercapita*PE_projected_comercial/1000
Q_projected_infiltration=Q_currentavginfiltration*climate_chage_factor
```

```
#Projected minimum flow rates (m3/day)
```

```
Q_projected_minimum_domestic=Q_projected_domestic*peak_minimum_factor
Q_projected_minimum_commercial=Q_projected_comercial*peak_minimum_factor
Q_projected_minimuminfiltration=Q_minimuminfiltration*climate_chage_factor
```

```
#Projected maximum flow rates (m3/day)
Q_projected_maximum_domestic=Q_projected_domestic*peak_maximum_factor
Q_projected_maximum_commercial=Q_projected_comercial*peak_maximum_factor
Q_projected_maximuminfiltration=Q_maximuminfiltration*climate_chage_factor
```

```
#Total Projected flow rates (m3/day)
Q_total_projected_maximum=Q_projected_maximum_domestic+Q_projected_maximum_commercial+Q_projected_maximuminfiltration
Q_total_projected_minimum=Q_projected_minimum_domestic+Q_projected_minimum_commercial+Q_projected_minimuminfiltration
Q_total_projected_average=Q_projected_domestic+Q_projected_comercial+Q_projected_infiltration
```

```
#Total current flow rates (m3/day)
Q_total_current_maximum=Q_maximumdomestic+Q_maximumcommercial+Q_maximuminfiltration
Q_total_current_minimum=Q_minimumdomestic+Q_minimumcommercial+Q_minimuminfiltration
Q_total_current_average=Q_currentavgdomestic+Q_currentavgcommercial+Q_currentavginfiltration
```

```
print (PE_projected_domestic)
print (PE_projected_comercial)
```

```
data_current = {
    'Contribution type': ['Domestic', 'Commercial', 'Infiltration', 'Total'],
    'Population': [PE_domestic, PE_commercial, '--', '--'],
    'Minimum flow rate (m3/day)': [Q_minimumdomestic, Q_minimumcommercial, Q_minimuminfiltration, Q_total_current_minimum],
    'Average flow rate (m3/day)': [Q_currentavgdomestic, Q_currentavgcommercial, Q_currentavginfiltration, Q_total_current_average],
    'Maximum flow rate (m3/day)': [Q_maximumdomestic, Q_maximumcommercial, Q_maximuminfiltration, Q_total_current_maximum],
}
```

```
df1 = pd.DataFrame(data_current)
display(df1)
```

```
data_projected = {
    'Contribution type': ['Domestic ', 'Commercial', 'Infiltration', 'Total'],
```

```

'Population':[PE_projected_domestic,PE_projected_comercial,'--','--'],
'Minimum flow rate (m3/day)': [Q_projected_minimum_domestic, Q_projected_minimum_commercial, Q_projected_minimuminfiltration, Q_total_pi
'Average flow rate (m3/day)': [Q_projected_domestic,Q_projected_comercial, Q_projected_infiltration,Q_total_projected_average],
'Maximum flow rate (m3/day)': [Q_projected_minimum_domestic, Q_projected_minimum_commercial, Q_projected_minimuminfiltration, Q_total_pi
}
df2 = pd.DataFrame(data_projected)
display(df2)

```

↔ 508715
42584

	Contribution type	Population	Minimum flow rate (m3/day)	Average flow rate (m3/day)	Maximum flow rate (m3/day)	
0	Domestic	340000	16524	55080	104652	
1	Commercial	28461	2391	7969	15141	
2	Infiltration	--	11657	11657	11657	
3	Total	--	30572	74706	131450	
	Contribution type	Population	Minimum flow rate (m3/day)	Average flow rate (m3/day)	Maximum flow rate (m3/day)	
0	Domestic	508715	24724	82412	24724	
1	Commercial	42584	3577	11924	3577	
2	Infiltration	--	13988	13988	13988	
3	Total	--	42289	108324	193226	

Next
steps:

Generate code with df1

View recommended plots

New interactive sheet

Generate code with df2

View recommended plots

New inter

✓ Design of screens - Preliminary treatment

5.4.3 <https://www.youtube.com/watch?v=2WwA7Mdrm8k>

#Preliminary treatment design:

#Velocity should be between 0.4 and 0.9 m/s

#Screen Width is recommended between 0.6 and 1.1 m

velocity_screen=0.90 #m/s, this value the first approximation

cd=0.84 #dimensionless, this is a value of discharge coefficient for the screen

gravity=9.81 #m/s^2

width_between_bars=0.009 #m, given that it need to trap gross pollutants >10 mm

bar_width=0.007 #m, depends on commercial availability. 8mm is within the commercial range.

velocity_upstream_screen=velocity_screen*width_between_bars/(width_between_bars+bar_width) #m/s

print ('The velocity upstream the screen is: '+str(velocity_upstream_screen))

head_loss_screen=round(1/(2*gravity*cd**2)*(velocity_screen**2-velocity_upstream_screen**2),2)

print ('the head loss screen is: '+str(head_loss_screen))

#Design of the screen channel

flow_depth_downstream=1 #m during desing peak flow.

flow_depth_upstream=flow_depth_downstream+head_loss_screen

print ('the flow depth upstream is: '+str(flow_depth_upstream))

wet_area=round((Q_total_projected_maximum/86400)/velocity_upstream_screen,2) #Flow converted into m^3/s, area in m^2

print ('The weat area is: '+str(wet_area)+' m^2')

width_channel_screen = (wet_area / flow_depth_upstream)

Round up to the nearest multiple of 0.1

width_channel_screen_rounded = round(math.ceil(width_channel_screen / 0.2) * 0.2,2) #m

print ('The suitable width channel is: '+str(width_channel_screen_rounded)+' m')

if (width_channel_screen_rounded>1.1):

screen_number=math.ceil(width_channel_screen_rounded/1.1)

if not 0.3<screen_number<1.1:

screen_number=math.ceil(screen_number)

else:

```

print('it is too narrow')

print ('The number of screen should be: '+str(screen_number))

velocity_upstream_screen_projected_maximum=Q_total_projected_maximum/(86400*flow_depth_upstream*width_channel_screen_rounded) #m/s
print ('The velocity upstream the screen is: '+str(velocity_upstream_screen_projected_maximum)+ ' m/s')

#It need to comply with the minimum velocity with the minimum current flow. In this case, we want to find the
#downstream depth flow controlled by a hydraulic structure since this is not provided in the briefing.

velocity_upstream_screen_current_average=0.5 #m/s desired velocity

flow_depth_downstream_current_average=round((Q_total_current_average/86400)/(velocity_upstream_screen_current_average*width_channel_screen_r

print ('The depth flow controlled by a hydraulic structure should be less than '+str(flow_depth_downstream_current_average)+' m in order to

print( )
print( )

import sympy as sp

# Velocity with a 50 percent blockage and the average current flow

# 1) define symbols and constants
width_screen=width_channel_screen_rounded/screen_number
print ('The width of the screen is: '+str(width_screen))
v1_average = sp.symbols('v1', real=True)
h2_average = flow_depth_downstream_current_average

# 2) express h1(v1) and Δh(v1) from your derivation
vsc_average= (Q_total_current_average/86400/screen_number)/(h2_average*((width_screen-bar_width)/(bar_width+width_between_bars)*width_betwe
print ('The velocity screen chamber average flow is: '+str(vsc_average)+'m/s')
v2_average = (Q_total_current_average/screen_number/86400)/(h2_average*width_screen)
print ('The velocity downstream is: '+str(v2_average)+'m/s')
h1_average= h2_average*v2_average/v1_average #
vsc_coefficient_average=(width_between_bars+bar_width)/(width_between_bars*0.5)
delta = ((vsc_coefficient_average**2-1**2)* v1_average**2)/(2*gravity*cd**2) # Δh =

# 3) build the Bernoulli equation:  $h_1 + v_1^2/(2g) = h_2 + v_2^2/(2g) + \Delta h$ 
eq_average = h1_average + v1_average**2/(2*gravity) - (h2_average + v2_average**2/(2*gravity) + delta)

# 4) solve for v1_average
solutions_average = sp.solve(eq_average, v1_average)
print('The velocity with a 50 percent blockage and the average current flow is: '+str(solutions_average)+' m/s')

# Velocity with a 50 percent blockage and the peak projected flow

v1_peak = sp.symbols('v1', real=True)
h2_peak = flow_depth_downstream
# 2) express h1(v1) and Δh(v1) from your derivation
vsc_peak= (Q_total_projected_maximum/86400/screen_number)/(flow_depth_upstream*((width_screen-bar_width)/(bar_width+width_between_bars)*wid
print ('The velocity screen chamber peak flow is: '+str(vsc_peak)+'m/s')
v2_peak = (Q_total_projected_maximum/screen_number/86400)/(flow_depth_downstream*width_screen)
print ('The velocity downstream is: '+str(v2_peak)+'m/s')
h1_peak = h2_peak*v2_peak/v1_peak
vsc_coefficient_peak=(width_between_bars+bar_width)/(width_between_bars*0.5)
delta = ((vsc_coefficient_peak**2-1**2)* v1_peak**2)/(2*gravity*cd**2) # Δh =

# 3) build the Bernoulli equation:  $h_1 + v_1^2/(2g) = h_2 + v_2^2/(2g) + \Delta h$ 
eq_peak = h1_peak + v1_peak**2/(2*gravity) - (h2_peak + v2_peak**2/(2*gravity) + delta)

# 4) solve for v1_average
solutions_peak = sp.solve(eq_peak, v1_peak)
print('The velocity with a 50 percent blockage and the peak projected flow is: '+str(solutions_peak)+' m/s')

```

 The velocity upstream the screen is: 0.50625
 the head loss screen is: 0.04
 the flow depth upstream is: 1.04
 The weat area is: 4.42 m2
 The suitable width channel is: 4.4 m
 The number of screen should be: 4
 The velocity upstream the screen is: 0.48872530644991574 m/s

The depth flow controlled by a hydraulic structure should be less than 0.35 m in order to comply with the minimum velocity with the ave

The width of the screen is: 1.1
 The velocity screen chamber average flow is: 1.00454982349869m/s
 The velocity downstream is: 0.5614634439634439m/s
 The velocity with a 50 percent blockage and the average current flow is: [0.399364243398226] m/s
 The velocity screen chamber peak flow is: 0.8744094127801683m/s
 The velocity downstream is: 0.5082743187079123m/s
 The velocity with a 50 percent blockage and the peak projected flow is: [0.436722737979358] m/s

✓ Equalization basin

9.7.6. Equalization Sustainability and Resource Recovery Equalization operations have little resource recovery potential. However, they could provide significant sustainable features to a WRRF, as peak demand loadings and their associated larger energy requirements can be avoided when equalization is practiced.

Botero, Lucas, Joel C. Rife, Kendra D. Sveum, and Alex Szerwinski. 2018. "Equalization." Chap. 9.7 in Design of Water Resource Recovery Facilities. 6th ed., edited by The Water Environment Federation (WEF). McGraw-Hill Education: New York, Chicago, San Francisco, Athens, London, Madrid, Mexico City, Milan, New Delhi, Singapore, Sydney, Toronto.

<https://www.accessengineeringlibrary.com/content/book/9781260031188/toc-chapter/chapter9/section/section101>

CURRENT

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display

Q_new_peak = Q_total_current_average # m³/day, this becomes the new peak flow rate taken from the calculation of the primary treatment.

# Create data for the table
data = {
    'Time of Day': [f"{i} to {i+1}" for i in range(0, 24)],
    'Q average current flow (m³/hr)': [Q_total_current_average/24] * 24, # Constant average flow
    'Q new peak (m³/hr)': [Q_new_peak/24] * 24, # Constant average flow
}

# Flow distribution that follows a pattern (low at night, high during day)
# This values are taken from Daily fluctuation of peaking factors graph given in the briefing.
flow_coefficients = [
    0.6, 0.5, 0.4, 0.3, 0.3, 0.4, 0.6, 0.7,
    0.8, 0.9, 1, 1, 1.1, 1.2, 1.3, 1.4,
    1.6, 1.7, 1.9, 1.8, 1.6, 1.2, 0.9, 0.7
]

# Calculate actual flow values by multiplying coefficients by the average
flow_values_current = [coef * Q_total_current_average/24 for coef in flow_coefficients]
data['Flow distribution current Q (m³/hr)'] = flow_values_current[:24]

# Calculate the difference
data['Difference (Flow current-Q new peak) (m³/hr)'] = [q - data['Q new peak (m³/hr)'][0] for q in data['Flow distribution current Q (m³/hr)']]
data['Difference (Flow current-Qavg) (m³/hr)'] = [q - data['Q average current flow (m³/hr)'][0] for q in data['Flow distribution current Q (m³/hr)']]

# Initialize storage calculations
data['Amount to Storage (m³)'] = [0] * 24
data['Amount from Storage (m³)'] = [0] * 24
data['Running Total in Storage (m³)'] = [0] * 24

# Calculate storage values
running_total = 0
for i in range(24):
    diff1 = data['Difference (Flow current-Q new peak) (m³/hr)'][i]
    diff2 = data['Difference (Flow current-Qavg) (m³/hr)'][i]

    # If flow > avg, we add to storage
    if diff2 > 0 and diff1 > 0:
        data['Amount to Storage (m³)'][i] = diff1
        data['Amount from Storage (m³)'][i] = 0
        running_total += diff1
    # If flow < avg, we take from storage
    else:
        if running_total == 0:
```

```

    data['Amount to Storage (m³)'][i] = 0
    data['Amount from Storage (m³)'][i] = 0
    running_total += 0
else:
    data['Amount to Storage (m³)'][i] = 0
    data['Amount from Storage (m³)'][i] = abs(diff2)
    running_total -= abs(diff2)

data['Running Total in Storage (m³)'][i] = running_total

# Create DataFrame
df = pd.DataFrame(data)

# Format the DataFrame for better display
# Format time of day column
df['Time of Day'] = [f"{i:2d} to {i+1:2d}" for i in range(0, 24)]

# Apply formatting to numeric columns to match the table
pd.options.display.float_format = '{:.0f}'.format

# Create styled table
styled_df = df.style.set_table_styles([
    {'selector': 'th', 'props': [('color', 'black'),
                                  ('font-weight', 'bold'),
                                  ('border', '1px solid #ddd'),
                                  ('padding', '5px')]},
    {'selector': 'td', 'props': [('border', '1px solid #ddd'),
                                  ('padding', '5px')]},
    {'selector': '', 'props': [('border-collapse', 'collapse'),
                                ('width', '100%'),
                                ('font-size', '14px')]}
])

# Group headers
header_style = {
    'selector': 'th.col_heading',
    'props': [('background-color', '#0a6e3c'),
              ('color', 'white'),
              ('text-align', 'center'),
              ('font-weight', 'bold'),
              ('border', '1px solid black'),
              ('padding', '5px')]
}

# Display the styled table
display(styled_df)

# If you want to save to Excel
df.to_excel("water_flow_table.xlsx", index=False)

# Create a visualization of the data
plt.figure(figsize=(12, 8))

# Plot flow distribution vs average flow
plt.subplot(2, 1, 1)
plt.plot(range(24), df['Flow distribution current Q (m³/hr)'], 'b-', label='Flow Distribution')
plt.plot(range(24), df['Q average current flow (m³/hr)'], 'g--', label='Current average Flow')
plt.xlabel('Hour of Day')
plt.ylabel('Flow Rate (m³/hr)')
plt.title('Flow Distribution vs Average Flow')
plt.grid(True)
plt.legend()

# Plot storage metrics
plt.subplot(2, 1, 2)
plt.bar(range(24), df['Amount to Storage (m³)'], color='green', label='Amount to Storage')
plt.bar(range(24), df['Amount from Storage (m³)'], color='red', label='Amount from Storage')
plt.plot(range(24), df['Running Total in Storage (m³)'], 'k-', label='Running Total')
plt.xlabel('Hour of Day')
plt.ylabel('Storage Volume (m³)')
plt.title('Storage Metrics')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

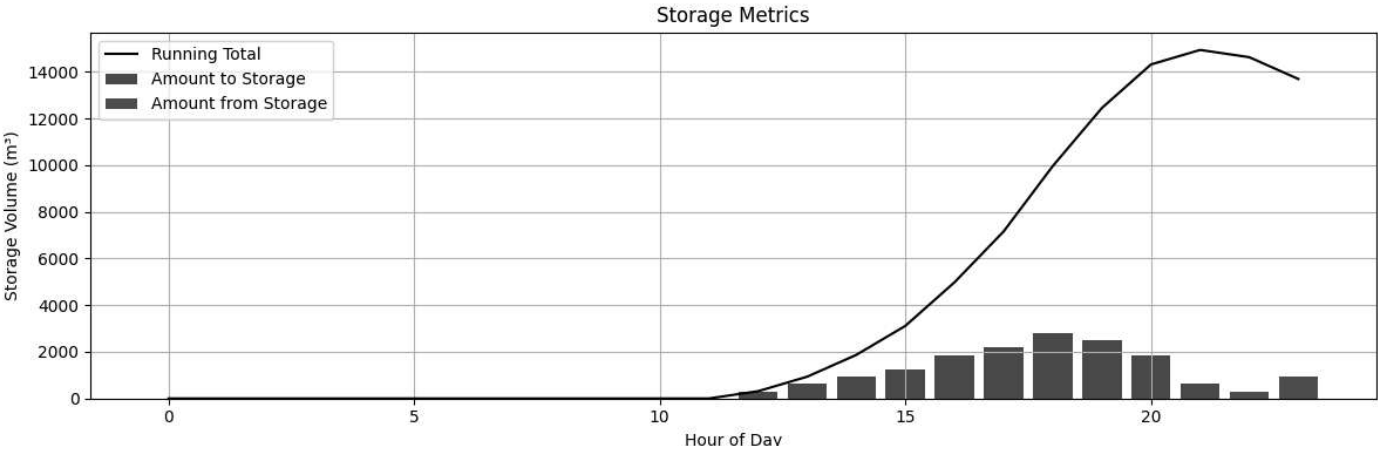
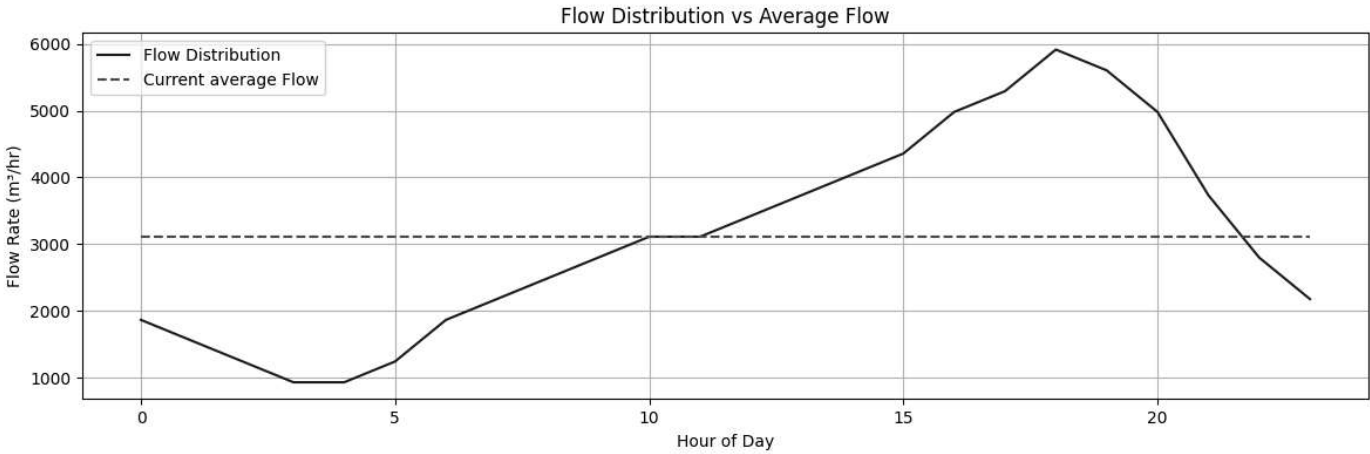
```

```
#Include adquate safety factors
```

```
safety_factor=1.2 #We need to justify this.  
required_volume_equalizer=safety_factor*max(df['Running Total in Storage (m³)'])  
print (max(df['Running Total in Storage (m³)']))  
rounded_required_volume_equalizer=round(required_volume_equalizer,-3) #m3  
print ('The required volume of the equalizer is: '+str(required_volume_equalizer))
```



	Time of Day	Q average current flow (m³/hr)	Q new peak (m³/hr)	Flow distribution current Q (m³/hr)	Difference (Flow current-Q new peak) (m³/hr)	Difference (Flow current- Qavg) (m³/hr)	Amount to Storage (m³)	Amount from Storage (m³)	Running Total in Storage (m³)
0	0 to 1	3112.753333	3112.753333	1867.652000	-1245.101333	-1245.101333	0.000000	0.000000	0.000000
1	1 to 2	3112.753333	3112.753333	1556.376667	-1556.376667	-1556.376667	0.000000	0.000000	0.000000
2	2 to 3	3112.753333	3112.753333	1245.101333	-1867.652000	-1867.652000	0.000000	0.000000	0.000000
3	3 to 4	3112.753333	3112.753333	933.826000	-2178.927333	-2178.927333	0.000000	0.000000	0.000000
4	4 to 5	3112.753333	3112.753333	933.826000	-2178.927333	-2178.927333	0.000000	0.000000	0.000000
5	5 to 6	3112.753333	3112.753333	1245.101333	-1867.652000	-1867.652000	0.000000	0.000000	0.000000
6	6 to 7	3112.753333	3112.753333	1867.652000	-1245.101333	-1245.101333	0.000000	0.000000	0.000000
7	7 to 8	3112.753333	3112.753333	2178.927333	-933.826000	-933.826000	0.000000	0.000000	0.000000
8	8 to 9	3112.753333	3112.753333	2490.202667	-622.550667	-622.550667	0.000000	0.000000	0.000000
9	9 to 10	3112.753333	3112.753333	2801.478000	-311.275333	-311.275333	0.000000	0.000000	0.000000
10	10 to 11	3112.753333	3112.753333	3112.753333	0.000000	0.000000	0.000000	0.000000	0.000000
11	11 to 12	3112.753333	3112.753333	3112.753333	0.000000	0.000000	0.000000	0.000000	0.000000
12	12 to 13	3112.753333	3112.753333	3424.028667	311.275333	311.275333	311.275333	0.000000	311.275333
13	13 to 14	3112.753333	3112.753333	3735.304000	622.550667	622.550667	622.550667	0.000000	933.826000
14	14 to 15	3112.753333	3112.753333	4046.579333	933.826000	933.826000	933.826000	0.000000	1867.652000
15	15 to 16	3112.753333	3112.753333	4357.854667	1245.101333	1245.101333	1245.101333	0.000000	3112.753333
16	16 to 17	3112.753333	3112.753333	4980.405333	1867.652000	1867.652000	1867.652000	0.000000	4980.405333
17	17 to 18	3112.753333	3112.753333	5291.680667	2178.927333	2178.927333	2178.927333	0.000000	7159.332667
18	18 to 19	3112.753333	3112.753333	5914.231333	2801.478000	2801.478000	2801.478000	0.000000	9960.810667
19	19 to 20	3112.753333	3112.753333	5602.956000	2490.202667	2490.202667	2490.202667	0.000000	12451.013333
20	20 to 21	3112.753333	3112.753333	4980.405333	1867.652000	1867.652000	1867.652000	0.000000	14318.665333
21	21 to 22	3112.753333	3112.753333	3735.304000	622.550667	622.550667	622.550667	0.000000	14941.216000
22	22 to 23	3112.753333	3112.753333	2801.478000	-311.275333	-311.275333	0.000000	311.275333	14629.940667
23	23 to 24	3112.753333	3112.753333	2178.927333	-933.826000	-933.826000	0.000000	933.826000	13696.114667



14941.215999999999

The required volume of the equalizer is: 17929.459199999998

PROJECTED

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display

Q_new_peak = 4768.93*24 # m³/day, this becomes the new peak flow rate taken from the calculation of the primary treatment.

# Create data for the table
data = {
    'Time of Day': [f"{i}" to {i+1}" for i in range(0, 24)],
    'Q average projected flow (m³/hr)': [Q_total_projected_average/24] * 24, # Constant average flow
    'Q new peak (m³/hr)': [Q_new_peak/24] * 24, # Constant average flow
}

# Flow distribution that follows a pattern (low at night, high during day)
# This values are taken from Daily fluctuation of peaking factors graph given in the briefing.
flow_coefficients = [
    0.6, 0.5, 0.4, 0.3, 0.3, 0.4, 0.6, 0.7,
    0.8, 0.9, 1, 1, 1.1, 1.2, 1.3, 1.4,
    1.6, 1.7, 1.9, 1.8, 1.6, 1.2, 0.9, 0.7
]

# Calculate actual flow values by multiplying coefficients by the average
flow_values_projected = [coef * Q_total_projected_average/24 for coef in flow_coefficients]
data['Flow distribution projected Q (m³/hr)'] = flow_values_projected[:24]

# Calculate the difference
data['Difference (Flow projected-Q new peak) (m³/hr)'] = [q - data['Q new peak (m³/hr)'][0] for q in data['Flow distribution projected Q (m³/hr)']]
data['Difference (Flow projected-Qavg) (m³/hr)'] = [q - data['Q average projected flow (m³/hr)'][0] for q in data['Flow distribution projected Q (m³/hr)']]

# Initialize storage calculations
data['Amount to Storage (m³)'] = [0] * 24
data['Amount from Storage (m³)'] = [0] * 24
data['Running Total in Storage (m³)'] = [0] * 24

# Calculate storage values
running_total = 0
for i in range(24):
    diff1 = data['Difference (Flow projected-Q new peak) (m³/hr)'][i]
    diff2 = data['Difference (Flow projected-Qavg) (m³/hr)'][i]

    # If flow > avg, we add to storage
    if diff2 > 0 and diff1 > 0:
        data['Amount to Storage (m³)'][i] = diff1
        data['Amount from Storage (m³)'][i] = 0
        running_total += diff1
    # If flow < avg, we take from storage
    else:
        if running_total == 0:
            data['Amount to Storage (m³)'][i] = 0
            data['Amount from Storage (m³)'][i] = 0
            running_total += 0
        else:
            data['Amount to Storage (m³)'][i] = 0
            data['Amount from Storage (m³)'][i] = abs(diff2)
            running_total -= abs(diff2)

    data['Running Total in Storage (m³)'][i] = running_total

# Create DataFrame
df2 = pd.DataFrame(data)

# Format the DataFrame for better display
# Format time of day column
df2['Time of Day'] = [f"{i:2d}" to {i+1:2d}" for i in range(0, 24)]

# Apply formatting to numeric columns to match the table
pd.options.display.float_format = '{:.0f}'.format
```

```

# Create styled table
styled_df2 = df2.style.set_table_styles([
    {'selector': 'th', 'props': [('color', 'black'),
                                  ('font-weight', 'bold'),
                                  ('border', '1px solid #ddd'),
                                  ('padding', '5px')]},
    {'selector': 'td', 'props': [('border', '1px solid #ddd'),
                                  ('padding', '5px')]},
    {'selector': '', 'props': [('border-collapse', 'collapse'),
                                ('width', '100%'),
                                ('font-size', '14px')]}
])

# Group headers
header_style = {
    'selector': 'th.col_heading',
    'props': [('background-color', '#0a6e3c'),
               ('color', 'white'),
               ('text-align', 'center'),
               ('font-weight', 'bold'),
               ('border', '1px solid black'),
               ('padding', '5px')]
}

# Display the styled table
display(styled_df2)

# If you want to save to Excel
df2.to_excel("water_flow_table.xlsx", index=False)

# Create a visualization of the data
plt.figure(figsize=(12, 8))

# Plot flow distribution vs average flow
plt.subplot(2, 1, 1)
plt.plot(range(24), df2['Flow distribution projected Q (m³/hr)'], 'y-', label='Projected Flow Distribution')
plt.plot(range(24), df2['Q new peak (m³/hr)'], 'r--', label='New peak Flow')
plt.plot(range(24), df2['Q average projected flow (m³/hr)'], 'c--', label='Projected average Flow')
plt.plot(range(24), df2['Q average current flow (m³/hr)'], 'g--', label='Current average Flow')
plt.xlabel('Hour of Day')
plt.ylabel('Flow Rate (m³/hr)')
plt.title('Flow Distribution vs Average Flow')
plt.grid(True)
plt.legend()

# Plot storage metrics
plt.subplot(2, 1, 2)
plt.bar(range(24), df2['Amount to Storage (m³)'], color='green', label='Amount to Storage')
plt.bar(range(24), df2['Amount from Storage (m³)'], color='red', label='Amount from Storage')
plt.plot(range(24), df2['Running Total in Storage (m³)'], 'k-', label='Running Total')
plt.xlabel('Hour of Day')
plt.ylabel('Storage Volume (m³)')
plt.title('Storage Metrics')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

#Include adequate safety factors

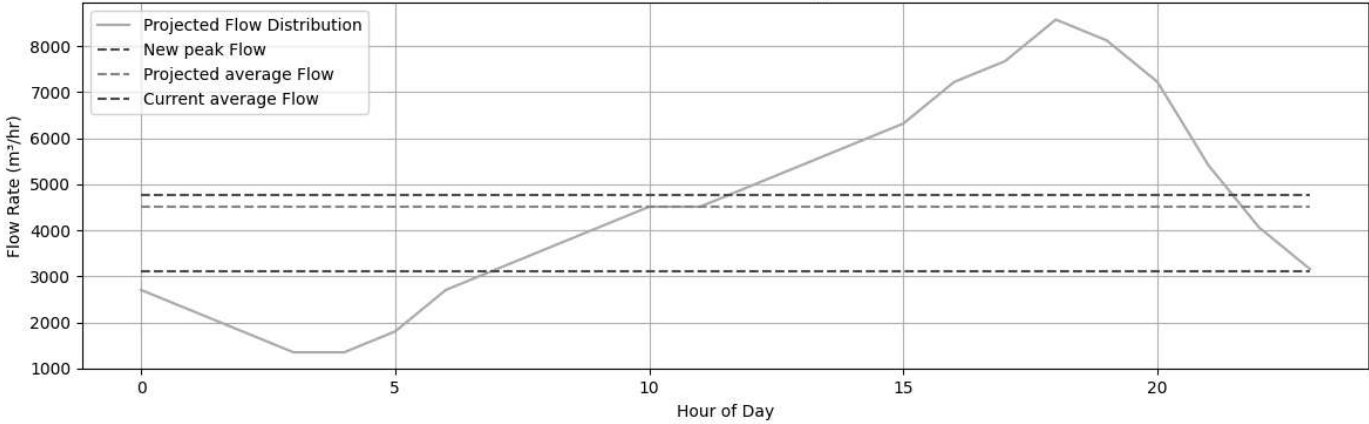
safety_factor=1.1 #We need to justify this.
required_volume_equalizer=safety_factor*max(df2['Running Total in Storage (m³)'])
print(max(df2['Running Total in Storage (m³)']))
rounded_required_volume_equalizer=round(required_volume_equalizer,-3) #m3
print('The required volume of the equalizer is: '+str(required_volume_equalizer))

```

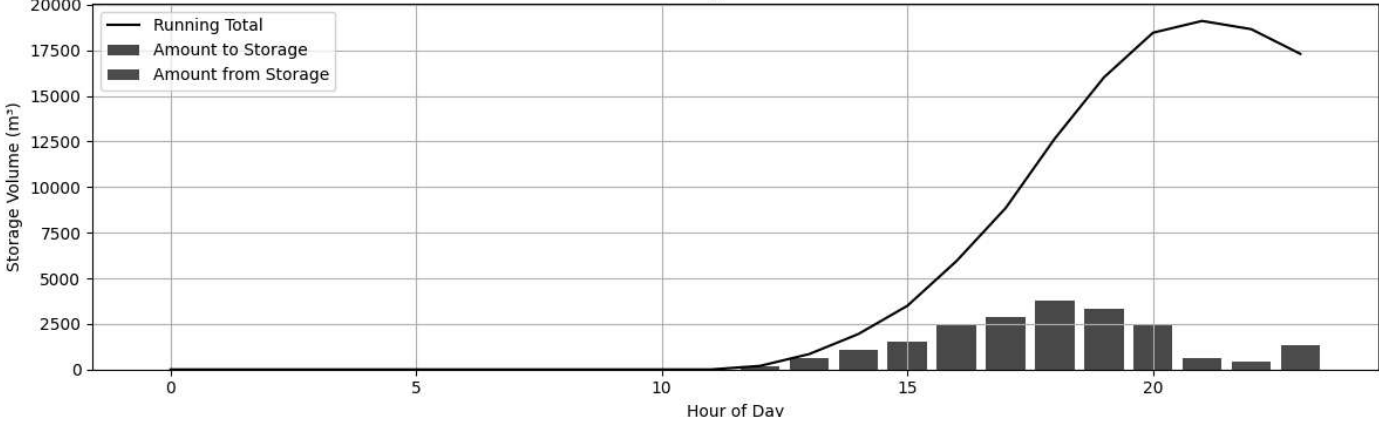


	Time of Day	Q average projected flow (m³/hr)	Q new peak (m³/hr)	Flow distribution projected Q (m³/hr)	Difference (Flow projected-Q new peak) (m³/hr)	Difference (Flow projected- Qavg) (m³/hr)	Amount to Storage (m³)	Amount from Storage (m³)	Running Total in Storage (m³)
0	0 to 1	4513.489583	4768.930000	2708.093750	-2060.836250	-1805.395833	0.000000	0.000000	0.000000
1	1 to 2	4513.489583	4768.930000	2256.744792	-2512.185208	-2256.744792	0.000000	0.000000	0.000000
2	2 to 3	4513.489583	4768.930000	1805.395833	-2963.534167	-2708.093750	0.000000	0.000000	0.000000
3	3 to 4	4513.489583	4768.930000	1354.046875	-3414.883125	-3159.442708	0.000000	0.000000	0.000000
4	4 to 5	4513.489583	4768.930000	1354.046875	-3414.883125	-3159.442708	0.000000	0.000000	0.000000
5	5 to 6	4513.489583	4768.930000	1805.395833	-2963.534167	-2708.093750	0.000000	0.000000	0.000000
6	6 to 7	4513.489583	4768.930000	2708.093750	-2060.836250	-1805.395833	0.000000	0.000000	0.000000
7	7 to 8	4513.489583	4768.930000	3159.442708	-1609.487292	-1354.046875	0.000000	0.000000	0.000000
8	8 to 9	4513.489583	4768.930000	3610.791667	-1158.138333	-902.697917	0.000000	0.000000	0.000000
9	9 to 10	4513.489583	4768.930000	4062.140625	-706.789375	-451.348958	0.000000	0.000000	0.000000
10	10 to 11	4513.489583	4768.930000	4513.489583	-255.440417	0.000000	0.000000	0.000000	0.000000
11	11 to 12	4513.489583	4768.930000	4513.489583	-255.440417	0.000000	0.000000	0.000000	0.000000
12	12 to 13	4513.489583	4768.930000	4964.838542	195.908542	451.348958	195.908542	0.000000	195.908542
13	13 to 14	4513.489583	4768.930000	5416.187500	647.257500	902.697917	647.257500	0.000000	843.166042
14	14 to 15	4513.489583	4768.930000	5867.536458	1098.606458	1354.046875	1098.606458	0.000000	1941.772500
15	15 to 16	4513.489583	4768.930000	6318.885417	1549.955417	1805.395833	1549.955417	0.000000	3491.727917
16	16 to 17	4513.489583	4768.930000	7221.583333	2452.653333	2708.093750	2452.653333	0.000000	5944.381250
17	17 to 18	4513.489583	4768.930000	7672.932292	2904.002292	3159.442708	2904.002292	0.000000	8848.383542
18	18 to 19	4513.489583	4768.930000	8575.630208	3806.700208	4062.140625	3806.700208	0.000000	12655.083750
19	19 to 20	4513.489583	4768.930000	8124.281250	3355.351250	3610.791667	3355.351250	0.000000	16010.435000
20	20 to 21	4513.489583	4768.930000	7221.583333	2452.653333	2708.093750	2452.653333	0.000000	18463.088333
21	21 to 22	4513.489583	4768.930000	5416.187500	647.257500	902.697917	647.257500	0.000000	19110.345833
22	22 to 23	4513.489583	4768.930000	4062.140625	-706.789375	-451.348958	0.000000	451.348958	18658.996875
23	23 to 24	4513.489583	4768.930000	3159.442708	-1609.487292	-1354.046875	0.000000	1354.046875	17304.950000

Flow Distribution vs Average Flow



Storage Metrics



```
19110.345833333333
```

```
The required volume of the equalizer is: 21021.380416666667
```

✓ Secondary Treatment

First Iteration - Using Conventional process

```
#First, we will calculate the influent BOD5 concentration from the primary clarifier knowing that the BOD5 removal efficiency is
#93% and the effluent BOD5 concentration is 18 mg/L. This is the maximum value that we can have in the effluent and the maximum
# efficiency that we can have in the secondary clarifier.

BOD_removal_efficiency=0.9318 #dimensionless, this is the range of maximum value that we can have in the secondary clarifier for the conventional process
print ('The BOD removal efficiency is: '+str(BOD_removal_efficiency))
BOD_required_to_effluent=18 #mg/L

#After that we calculate the BOD5 concentration in the primary clarifier outflow using the following equation:
BOD_primary_clarifier=round(BOD_required_to_effluent/(1-BOD_removal_efficiency),2) #mg/L
print ('BOD primary clarifier is: ' +str(BOD_primary_clarifier)+ ' mg/L')

Q1=Q_new_peak #m3/d, This is the flow from the equalization tank upstream.

#MLSS value is taken since we have a 10 degree water temperature and it is the maximum value that we can have
#in the secondary clarifier according to Table 8-19 Metcalf & Eddy.

mixed_liquor_suspended_solids=3000 #mg/L(X), this should be between 1000-3000 mg/L for a conventional activated sludge process.

#Typical values of the yield coefficient and decay coefficient. Given that we have a 10 degree water temperature
#the values are modified using the Arrhenius equation.
yield_coefficient=0.45 #SS/mg BOD5 (Y)
decay_coefficient=0.04 #/day (Kd)
conversion_rate_constant=0.2 #/day (k)

#We want the SVI less than 120 mL/g, this is recommended to have a good settling sludge. Therefore, using 91 mL/g value
# we can calculate the return sludge concentration, reported by Chudoba et al (1973) in order to get a good mix of microbial community.

suspended_volumen_index=91 #mL/g
print ('SVI is: '+str(suspended_volumen_index)+' mL/g')

#Therefore, the return sludge concentration is:

return_sludge_suspended_solids=10**6/suspended_volumen_index #mg/L(XR),
print ('The return sludge concentration is: '+str(return_sludge_suspended_solids)+' mg/L')

#Using this SVI value, and having selected the MLSS value, we can calculate the volumen settled sludge.
volumen_settled_sludge=suspended_volumen_index*mixed_liquor_suspended_solids/1000 #v
print ('The volumen settled is: '+str(volumen_settled_sludge)+' mL/L')

#Knowing the volumen settled sludge, we can calculate the return sludge flow.
return_sludge_flow=(Q1*volumen_settled_sludge)/(1000-volumen_settled_sludge) #m3/d
print ('The return sludge is: '+str(return_sludge_flow)+' m3/d')

#Using the return sludge flow, we can calculate the return sludge ratio
return_sludge_ratio=return_sludge_flow/Q1 # this should be between 0.25-0.75 for a conventional activated sludge process.
print ('The return sludge ratio is: '+str(return_sludge_ratio))

food_to_microorganism_ratio=0.4 #This value depends on the process, for this case it is a conventional process.
#We select the minimum Try to minimize the aeration volumen. it could be 0.2-0.4 for a conventional activated sludge process according to 1

#The aeration volume is calculated using the BOD5 concentration from the primary clarifier, the flow rate, the food to microorganism ratio,
aeration_volumen=(BOD_primary_clarifier*Q1)/(food_to_microorganism_ratio*mixed_liquor_suspended_solids)
print ('aeration volumen is: ' +str(aeration_volumen)+ ' m3')

#The aeration time is calculated using the aeration volume and the flow rate.
aeration_time=(aeration_volumen/Q1)*24 #hours. This value should be between 4-8 hours for a conventional activated sludge process.
print ('aeration time is: ' +str(aeration_time)+ ' hours')

#This value should be between 300-700 g/m3/day for a conventional activated sludge process. Therefore, we need to select another process.
BOD_load=(BOD_primary_clarifier*Q1)/aeration_volumen #g/m3/day
print ('BOD load is: ' +str(BOD_load)+ ' g/m3/day')

#Even though I tried to vary the F/M ratio either the aeration time or BOD load are not in the range of the conventional activated sludge process
```



```

➡ The BOD removal efficiency is: 0.9318
BOD primary clarifier is: 263.93 mg/L
SVI is: 91 mL/g
The return sludge concentration is: 10989.010989010989 mg/L
The volumen settled is: 273.0 mL/L
The return sludge is: 42979.407647867956 m3/d
The return sludge ratio is: 0.375515818431912
aereation volumen is: 25173.273898000003 m3
aereation time is: 5.2786 hours
BOD load is: 1200.0 g/m3/day

```

Second Iteration - Using CMAS

#First, we will calculate the influent BOD5 concentration from the primary clarifier kwnowing that the BOD5 removal efficiency is #93% and the effluent BOD5 concentration is 18 mg/L. This is the maximum value that we can have in the effluent and the maximim # efficiency that we can have in the secondary clarifier.

```

BOD_removal_efficiency=0.9318 #dimensionless, this is the range of maximum value that we can have in the secondary clarifier for the CMAS prc
print ('The BOD removal efficiency is: '+str(BOD_removal_efficiency))
text=[str(round(BOD_removal_efficiency*100,1))]
keyword='py_BOD_removal_eff_sec'

```

```

BOD_required_to_effluent=18 #mg/L
text=[str(round(BOD_required_to_effluent,1))]
keyword='BOD_eff_sec'

```

```

BOD_primary_clarifier=round(BOD_required_to_effluent/(1-BOD_removal_efficiency),2) #mg/L
text=[str(round(BOD_primary_clarifier,1))]
keyword='py_BOD_in_sec'

```

```

print ('BOD primary clarifier is: ' +str(BOD_primary_clarifier)+ ' mg/L')

```

```

Q1=Q_new_peak #m3/d, This is the flow from the equalization tank upstream.
text=[str(round(Q1,1))]
keyword='py_Q_new_peak_secondary'

```

```

text=[str(round(Q_total_projected_average,1))]
keyword='py_Q_avg_sec'

```

#MLSS value is taken since we have a 10 degree water temperature and it is the maximum value that we can have #in the secondary clarifier according to Table 8-19 Metcalf & Eddy.

```

mixed_liquor_suspended_solids=4000 #mg/L(X), this shoould be between 2500-4000 mg/L for a CMAS activated sludge process.
text=[str(round(mixed_liquor_suspended_solids,1))]
keyword='py_MLSS_sec'
print ('The MLSS is: '+str(mixed_liquor_suspended_solids)+' mg/L')

```

```

#Typical values of the yield coefficient and decay coefficient. Given that we have a 10 degree water temperature
#the values are modified using the Arrhenius equation.
yield_coefficient=0.45 #SS/mg BOD5 (Y)
text=[str(round(yield_coefficient,2))]
keyword='py_Y_sec'

```

```

decay_coefficient=0.04 #/day (Kd)
text=[str(round(decay_coefficient,2))]
keyword='py_Kd_sec'

```

```

conversion_rate_constant=0.2 #/day (k), This a constant for BOD.
text=[str(round(conversion_rate_constant,1))]
keyword='py_k_sec'

```

#We want the SVI less than 120 mL/g, this is recommended to have a good settling sludge. Therefore, using 91 mL/g value # we can calculate the return sludge concentration, reported by Chudoba et al (1973) in order to get a good mix of microbial community

```

suspended_volumen_index=91 #mL/g
text=[str(round(suspended_volumen_index,1))]
keyword='py_SVI_sec'

print ('SVI is: '+str(suspended_volumen_index)+' mL/g')

#Therefore, the return sludge concentration is:

return_sludge_suspended_solids=round(10**6/suspended_volumen_index,2) #mg/L(XR),
text=[str(round(return_sludge_suspended_solids,1))]
keyword='py_X_R_sec'

print ('The return sludge concentration is: '+str(return_sludge_suspended_solids)+' mg/L')

#Using this SVI value, and havind selected the MLSS value, we can calculate the volumen settled sludge.
volumen_settled_sludge=suspended_volumen_index*mixed_liquor_suspended_solids/1000 #v
text=[str(round(volumen_settled_sludge,1))]
keyword='py_VolumeSettledSludge'

print ('The volumen settled is: '+str(volumen_settled_sludge)+' mL/L')

#Knowing the volumen settled sludge, we can calculate the return sludge flow.
return_sluge_flow=round(((Q1*volumen_settled_sludge)/(1000-volumen_settled_sludge),2) #m3/d
text=[str(round(return_sluge_flow,1))]
keyword='py_ReturnFlow_sec'

print ('The return sludge flow is: '+str(return_sluge_flow)+' m3/d')

#Using the return sludge flow, we can calculate the return sludge ratio
return_sludge_ratio=round(return_sluge_flow/Q1,2) # this should be between 0.25-1 for a CMAS activated sludge process.
print ('The return sludge ratio is: '+str(return_sludge_ratio))

food_to_microorganism_ratio=0.4 #This value depends on the process, for this case it is a CMAS process.
#We select the minimum Try to minimize the volumen. it could be 0.2-0.6 for a CMAS activated sludge process according to the table 8-19 Metca
#This value was initial supposed to be 0.6 but it was changed to 0.4 in order to get a aeration time.

text=[str(round(food_to_microorganism_ratio,1))]
keyword='py_F_M_sec'

#The aeration volume is calculated using the BOD5 concentration from the primary clarifier, the flow rate, the food to microorganism ratio, a
aeration_volumen=round((BOD_primary_clarifier*Q1)/(food_to_microorganism_ratio*mixed_liquor_suspended_solids),2)
text=[str(round(aeration_volumen,1))]
keyword='py_Aeration_Volume_sec'

print ('aeration volumen is: ' +str(aeration_volumen)+ ' m3')

#The aeration time is calculated using the aeration volume and the flow rate.
aeration_time=round((aeration_volumen/Q1)*24,2) #hours. This value should be between 3-6 hours for a CMAS activated sludge process.
text=[str(round(aeration_time,1))]
keyword='py_HRT_sec'

print ('aeration time is: ' +str(aeration_time) + ' hours')

#The organic loading rate, an essential performance parameter, is computed as follows:
#This value should be between 300-1600 g/m3/day for a CMAS activated sludge process.
BOD_load=round((BOD_primary_clarifier*Q1)/aeration_volumen,2) #g/m3/day
text=[str(round(BOD_load,1))]
keyword='py_Organic_load_rate_sec'

print ('BOD load is: ' +str(BOD_load)+ ' g/m3/day')

#Typically, kinetic coefficients for activated sludge processes are reported at 20 °C. To adapt these to our chosen design temperature (10 °C
specific_utilisation_ratio=round((food_to_microorganism_ratio*BOD_removal_efficiency),2) #dimensionless
text=[str(round(specific_utilisation_ratio,1))]
keyword='py_UtilizationRatio'

print ('specific utilisation ratio is: ' +str(specific_utilisation_ratio))

#Using these adjusted coefficients, the required sludge age is calculated. A safety factor of 1.3 is applied to accommodate peak organic load
#The sludge age should be between 5-15 days for a CMAS activated sludge process.
#Using these adjusted coefficients, the required sludge age is calculated. A safety factor of 1.3 is applied to accommodate peak organic load
sludge_age=round(1/(yield_coefficient*specific_utilisation_ratio-decay_coefficient),2) #days. This value should be between 5-15 days for a CM
text=[str(round(sludge_age,1))]
keyword='py_sludgeAge_sec'

```

```

print( 'sludge age is: ' +str(sludge_age)+ ' days ')

safety_factor_peak_org_loading=1.3 #This try to account the variation of BOD regardless the flow.

sludge_age_increased=round(sludge_age*safety_factor_peak_org_loading,2) #days. This value should be between 5-15 days for a CMAS activated sl
text=[str(round(sludge_age_increased,1))]
keyword='py_sludgeAgeAdj_sec'

print('sludge age increased is: ' +str(sludge_age_increased))

#Waste flow is calculated using the mixed liquor suspended solids concentration, the aeration volume, the return sludge suspended solids conc

flow_waste=(mixed_liquor_suspended_solids*aeration_volumen)/(return_sludge_suspended_solids*sludge_age)
text=[str(round(flow_waste,1))]
keyword='py_flowWaste'

print ('flow waste with a sludge age is: ' +str(flow_waste)+ ' m3/d')

flow_waste_increased=round((mixed_liquor_suspended_solids*aeration_volumen)/(return_sludge_suspended_solids*sludge_age_increased),2) #m3/d
text=[str(round(flow_waste_increased,1))]
keyword='py_flowWasteAdj'

print ('flow waste increased with a sludge age increased is: ' +str(flow_waste_increased)+ ' m3/d')

flow_effluent=round(Q1-flow_waste,2) #m3/d
text=[str(round(flow_effluent,1))]
keyword='py_effluentFlow'

print ('flow effluent with a sludge age is: ' +str(flow_effluent)+ ' m3/d')

flow_effluent_increased=round(Q1-flow_waste_increased,2) #m3/d
text=[str(round(flow_effluent_increased,1))]
keyword='py_effluentFlowAdj'

print ('flow effluent with a sludge age increased is: ' +str(flow_effluent_increased)+ ' m3/d')

print ()

maximum_volumen_cmas_tank=6*24*90 #m3, this is the maximum value that we can have in the tank.
print('maximum volumen cmass tank is: ' +str(maximum_volumen_cmas_tank)+ ' m3')

number_aeration_tanks=6
text=[str(round(number_aeration_tanks,1))]
keyword='py_NAerationTanks'

if aeration_volumen<maximum_volumen_cmas_tank:
    aeration_volumen_tank=maximum_volumen_cmas_tank
    print('aeration volumen tank is: ' +str(aeration_volumen)+ ' m3')
else:
    #I divide this value by 6 because the tank should be less than 3500 m3. This is to address the filamentous bulking problem.
    aeration_volumen_tank=round(aeration_volumen/number_aeration_tanks,2) #m3, this is the maximum value that we can have in the tank.
    print('each aeration volumen tank is: ' +str(aeration_volumen_tank)+ ' m3')

text=[str(round(aeration_volumen_tank,1))]
keyword='py_VolumePerAerationTank'

cmass_tank_depth=4.2 #m, it should between 3-6 m.
text=[str(round(cmass_tank_depth,1))]
keyword='py_aerationTankDepth'

print('cmass tank depth is: ' +str(cmass_tank_depth)+ ' m')

cmass_tank_width=4*cmass_tank_depth #m,
text=[str(round(cmass_tank_width,1))]
keyword='py_aerationTankWidth'

print('cmass tank width is: ' +str(cmass_tank_width)+ ' m')

cmass_tank_length=round(math.ceil(aeration_volumen_tank/(cmass_tank_depth*cmass_tank_width)/1)*1,2) #m
text=[str(round(cmass_tank_length,1))]
keyword='py_aerationTankLength'

print('cmass tank length is: ' +str(cmass_tank_length)+ ' m')

cmass tank calculated volume=cmass tank depth*cmass tank width*cmass tank length

```

```

cmass_tank_calculated_volume cmas_tank_oper cmas_tank_max cmas_tank_avg
text=[str(round(cmas_tank_calculated_volume,1))]
keyword='py_VolumeAerationTankFinal'

print( )

solids_loading_ratio_peak_flow=6 #kg/m2/h, this value is recommended between
text=[str(round(solids_loading_ratio_peak_flow,1))]
keyword='py_SLR_selected'

# 4-6 kg/m2/d for a average flow in the secondary clarifier, given that we have a similar peak and average flow due to equalisation basin.

area_secondary_clarifier=((Q1+return_sludge_ratio*Q1)*mixed_liquor_suspended_solids)/(solids_loading_ratio_peak_flow*24*1000) #m2, this is th
text=[str(round(area_secondary_clarifier,1))]
keyword='py_areaClarifier_sec'

print('area secondary clarifier is: ' +str(area_secondary_clarifier)+ ' m2')

solids_loading_ratio_average_flow=((Q_total_projected_average+return_sludge_ratio*Q1)*mixed_liquor_suspended_solids)/(area_secondary_clarifie
text=[str(round(solids_loading_ratio_average_flow,1))]
keyword='py_SLR_sec'

print('solids loading ratio average projected flow is: ' +str(solids_loading_ratio_average_flow)+ ' kg/m2/h')

print( )

#Aeration requirements
yield_observed_average=round(yield_coefficient/(1+decay_coefficient*sludge_age),2) #SS/mg BOD5 (Yobs)
text=[str(round(yield_observed_average,3))]
keyword='py_Yobs_sec'

rate_generation_biomass_Q_average=round(yield_observed_average*Q_total_projected_average*(BOD_primary_clarifier-BOD_required_to_effluent)/1000,2)
text=[str(round(rate_generation_biomass_Q_average,1))]
keyword='py_P_avg_sec'

print('Generation of biomass rate for Q average is: ' +str(rate_generation_biomass_Q_average)+ ' m3/d')

yield_observed_peak=yield_coefficient/(1+decay_coefficient*sludge_age_increased) #SS/mg BOD5 (Yobs)
text=[str(round(yield_observed_peak,3))]
keyword='py_Y_peaksec'

rate_generation_biomass_Q_peak=round(yield_observed_peak*Q_new_peak*(BOD_primary_clarifier-BOD_required_to_effluent)/1000,2) #Px
text=[str(round(rate_generation_biomass_Q_peak,1))]
keyword='py_P_peak_sec'

print('Generation of biomass rate for Q peak is: ' +str(rate_generation_biomass_Q_peak)+ ' m3/d')

f=1-math.exp(-5*conversion_rate_constant) #dimensionless
oxygen_demand_Q_peak=round((Q_new_peak*(BOD_primary_clarifier-BOD_required_to_effluent))/f/1000-1.42*rate_generation_biomass_Q_peak,2) #kg O2
text=[str(round(oxygen_demand_Q_peak,1))]
keyword='py_Oxygen_peak'

print('Oxygen demand for Q peak is: ' +str(oxygen_demand_Q_peak)+ ' kg O2/d')

oxygen_demand_Q_average=round((Q_total_projected_average*(BOD_primary_clarifier-BOD_required_to_effluent))/f/1000-1.42*rate_generation_biomass_Q_average,2)
text=[str(round(oxygen_demand_Q_average,1))]
keyword='py_OxygenAvg'

print('Oxygen demand for Q average is: ' +str(oxygen_demand_Q_average)+ ' kg O2/d')

air_required_peak=round(oxygen_demand_Q_peak/0.279/0.1*2/24,2) #kg O2/h, this is the value of the air saturation in the water.
text=[str(round(air_required_peak,1))]
keyword='py_air_peak'

print('Air required for Q peak is: ' +str(air_required_peak)+ ' m3 air/h')

air_required_typical=round(oxygen_demand_Q_average/0.279/0.1,2) #kg O2/h, this is the value of the air saturation in the water.
text=[str(round(air_required_typical,1))]
keyword='py_air_avg'

print('Air required for Q average is: ' +str(air_required_typical)+ ' m3 air/day')

```

 The BOD removal efficiency is: 0.9318
 BOD primary clarifier is: 263.93 mg/L

```

The MLSS is: 4000 mg/L
SVI is: 91 mL/g
The return sludge concentration is: 10989.01 mg/L
The volumen settled is: 364.0 mL/L
The return sludge flow is: 65505.3 m3/d
The return sludge ratio is: 0.57
aereation volumen is: 18879.96 m3
aereation time is: 3.96 hours
BOD load is: 1600.0 g/m3/day
specific utilisation ratio is: 0.37
sludge age is: 7.91 days
sludge age increased is: 10.28
flow waste with a sludge age is: 868.8123967771864 m3/d
flow waste increased with a sludge age increased is: 668.51 m3/d
flow effluent with a sludge age is: 113585.51 m3/d
flow effluent with a sludge age increased is: 113785.81 m3/d

maximum volumen cmas tank is: 12960 m3
each aereation volumen tank is: 3146.66 m3
cmas tank depth is: 4.2 m
cmas tank width is: 16.8 m
cmas tank length is: 45 m

area secondary clarifier is: 4991.480066666667 m2
solids loading ratio average projected flow is: 5.795298858651155 kg/m2/h

Generation of biomass rate for Q average is: 9057.62 m3/d
Generation of biomass rate for Q peak is: 8975.69 m3/d
Oxygen demand for Q peak is: 31783.61 kg O2/d
Oxygen demand for Q average is: 29398.47 kg O2/d
Air required for Q peak is: 94933.12 m3 air/h
Air required for Q average is: 1053708.6 m3 air/day

```

```
#Initial information Environmental impact assessment.
```

```
#Flow receiving waterway according to the group number given in the briefing.
```

```
flow_receiving_waterway=round((group_number**0.2*300000),3) #m3/day, this is a value taken from the briefing.
print('Flow receiving waterway: ' +str(flow_receiving_waterway)+ ' m3/day')
```

```
print('Flow discharge waterway: '+str(flow_effluent_increased)+' m3/day')
```

```
#BOD5 receiving waterway directly upstream from discharge point (Lr).
```

```
bod_upstream_receiving_waterway= 4 #mg/L, this is a value taken from the briefing.
```

```
saturation_dissolved_oxygen=8.5 #mg/L, this is a value taken from the briefing.
```

```
#Initial dissolve oxygen deficit in the receiving waterway (D0).
```

```
initial_disolved_oxygen_deficit=0.8 #mg/L, this is a value taken from the briefing.
```

```
#Reaeration rate constant base e (kr).
```

```
reaeration_rate_constant=0.71 #1/day, this is a value taken from the briefing.
```

```
#desoxygenation rate constant base e (kd).
```

```
desoxygenation_rate_constant=0.51 #1/day, this is a value taken from the briefing.
```

```
#stream velocity
```

```
stream_velocity=0.07 #m/s, this is a value taken from the briefing.
```

```
#disolved oxygen needs to be greater than 70% of saturation. According to ENVIRONMENT REFERENCE STANDARD for Victoria.
```

```
#This is for a surface waterway located in the north west of Melbourne.
```

```
minimum_dissolved_oxygen=round(0.7*saturation_dissolved_oxygen,2) #mg/L.
```

```
print('Minimum dissolved oxygen: ' +str(minimum_dissolved_oxygen)+' mg/L')
```

```
maximum_dissolved_oxygend_deficit=saturation_dissolved_oxygen-minimum_dissolved_oxygen #mg/L,
```

```
print('Maximum dissolved oxygen deficit: ' +str(maximum_dissolved_oxygend_deficit)+' mg/L')
```

```
max_bod_5_effluent=18 #mg/L, this is a value taken from EPA guidelines (Lw).
```

```
first_reaction_order_constant=0.23 #1/day, this is a value taken book reference Mcalf eddy pag 118-119.
```

```
bod_ultimate_effluent=round((max_bod_5_effluent/(1-math.exp(-first_reaction_order_constant*5))),2) #mg/L, this is a value taken from the br:
```

```
print('BOD ultimate effluent: ' +str(bod_ultimate_effluent)+' mg/L')
```

```
#This is the equation for the BOD ultimate mixing zone (Lo)
```

```
bod_ultimate_mixing_zone=round((flow_receiving_waterway*bod_upstream_receiving_waterway+flow_effluent_increased*bod_ultimate_effluent)/(flow_receiving_waterway+bod_upstream_receiving_waterway+flow_effluent_increased),2) #mg/L,
```

```
print('BOD ultimate mixing zone: ' +str(bod_ultimate_mixing_zone)+' mg/L')
```

```
#time at which DO will reach minimum level (tc).
```

```

time_minimum_dissolved_oxygen=1/(reaeration_rate_constant-desoxygenation_rate_constant)*math.log(reaeration_rate_constant/desoxygenation_ra

print('Time at which DO will reach minimum level: ' +str(time_minimum_dissolved_oxygen)+' days')

#In order to calculate the length of the minimum DO level, we use (tc) and the stream velocity (v).

lenght_minimum_DO_level=round(time_minimum_dissolved_oxygen*86400*stream_velocity,2) #m, this is a value taken from the briefing.
print('Length of the minimum DO level: ' +str(lenght_minimum_DO_level)+' m')

#Dissolve oxygen at the time of minimum level (D0t).
dissolved_oxygen_deficit_at_time_minimum_level=((desoxygenation_rate_constant*bod_ultimate_mixing_zone)/(reaeration_rate_constant-desoxygen:
print('Dissolved oxygen deficit at the time of minimum level: ' +str(dissolved_oxygen_deficit_at_time_minimum_level)+' mg/L')

if dissolved_oxygen_deficit_at_time_minimum_level>maximum_dissolved_oxygend_deficit:
    print('The dissolved oxygen deficit at the time of minimum level is greater than the maximum dissolved oxygen deficit.')
else:
    print('The dissolved oxygen deficit at the time of minimum level is less than the maximum dissolved oxygen deficit therefore it is suit:

#This calculation find the length where the dissolved oxygen is the same as the saturation dissolved oxygen.

```

```

→ Flow receiving waterway: 475467.958 m3/day
Flow discharge waterway: 113785.81 m3/day
Minimum dissolved oxygen: 5.95 mg/L
Maximum dissolved oxygen deficit: 2.55 mg/L
BOD ultimate effluent: 26.34 mg/L
BOD ultimate mixing zone: 7.33 mg/L
Time at which DO will reach minimum level: 1.4355555448886719 days
Length of the minimum DO level: 8682.24 m
Dissolved oxygen deficit at the time of minimum level: 2.5319454091655578 mg/L
The dissolved oxygen deficit at the time of minimum level is less than the maximum dissolved oxygen deficit therefore it is suitable.

```

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import fsolve

```

```

def solve_for_time(target_deficit, desoxygenation_rate_constant, reaeration_rate_constant,
                    bod_ultimate_mixing_zone, initial_disolved_oxygen_deficit):
    """
    Solves the Streeter-Phelps equation for time when dissolved oxygen deficit equals target value.

    The equation is:
    D(t) = (K1L0)/(K2-K1)[e-K1t - e-K2t] + D0e-K2t = target_deficit

    where:
    D(t) = dissolved oxygen deficit at time t
    K1 = desoxygenation_rate_constant
    K2 = reaeration_rate_constant
    L0 = bod_ultimate_mixing_zone
    D0 = initial_disolved_oxygen_deficit

    Args:
    target_deficit: Target dissolved oxygen deficit value
    desoxygenation_rate_constant: Deoxygenation rate constant (/day)
    reaeration_rate_constant: Reaeration rate constant (/day)
    bod_ultimate_mixing_zone: Ultimate BOD in mixing zone (mg/L)
    initial_disolved_oxygen_deficit: Initial dissolved oxygen deficit (mg/L)

    Returns:
    Time(s) at which DO deficit equals target value
    """
    # Define the Streeter-Phelps equation as a function to find roots
    def deficit_equation(t):
        # First term (BOD effect)
        bod_effect = (desoxygenation_rate_constant * bod_ultimate_mixing_zone) / (reaeration_rate_constant - desoxygenation_rate_constant)
        bod_effect *= (np.exp(-desoxygenation_rate_constant * t) - np.exp(-reaeration_rate_constant * t))

        # Second term (initial deficit effect)
        initial_effect = initial_disolved_oxygen_deficit * np.exp(-reaeration_rate_constant * t)

        # Complete equation (deficit at time t minus target deficit)
        return bod_effect + initial_effect - target_deficit

    # Find critical point (maximum deficit) first
    # This is important because deficit curve typically increases then decreases

```

```

t_values = np.linspace(0, 20, 1000)
deficits = np.zeros_like(t_values)

for i, t in enumerate(t_values):
    deficits[i] = deficit_equation(t) + target_deficit # Add target to get actual deficit

max_idx = np.argmax(deficits)
max_deficit = deficits[max_idx]
critical_time = t_values[max_idx]

print(f"Maximum deficit = {max_deficit:.4f} mg/L at t = {critical_time:.4f} days")

# Check if target deficit is achievable
if target_deficit > max_deficit:
    print(f"Target deficit ({target_deficit}) is higher than maximum deficit ({max_deficit:.4f})")
    print("No solution exists")
    return None

# Initialize results list
results = []

# Check if initial deficit equals target
if abs(initial_dissolved_oxygen_deficit - target_deficit) < 1e-6:
    results.append(0) # Time is 0 when initial equals target
    print(f"Initial deficit already equals target ({target_deficit})")

# Find time when deficit equals target (before critical point)
if initial_dissolved_oxygen_deficit < target_deficit < max_deficit:
    # Use numerical solver with initial guess halfway to critical point
    t_sol1 = fsolve(deficit_equation, critical_time/2)[0]

    # Verify solution
    if t_sol1 > 0 and abs(deficit_equation(t_sol1)) < 1e-6:
        results.append(t_sol1)
        print(f"Deficit reaches {target_deficit} at t = {t_sol1:.4f} days (rising limb)")

# Find time when deficit equals target (after critical point)
if target_deficit < max_deficit:
    # Use numerical solver with initial guess past critical point
    t_sol2 = fsolve(deficit_equation, critical_time + 2)[0]

    # Verify solution
    if t_sol2 > critical_time and abs(deficit_equation(t_sol2)) < 1e-6:
        results.append(t_sol2)
        print(f"Deficit reaches {target_deficit} at t = {t_sol2:.4f} days (falling limb)")

# Plot the results
plt.figure(figsize=(12, 8))

# Plot deficit curve
plt.plot(t_values, deficits, 'b-', linewidth=3, label='DO Deficit')

# Mark critical point
plt.plot([critical_time], [max_deficit], 'ro', markersize=10)
plt.annotate(f'Maximum: {max_deficit:.2f} mg/L',
            xy=(critical_time, max_deficit),
            xytext=(critical_time+1, max_deficit+0.1),
            arrowprops=dict(facecolor='red', shrink=0.05),
            fontsize=12)

# Mark target deficit
plt.axhline(y=target_deficit, color='g', linestyle='--', alpha=0.7,
            label=f'Target Deficit = {target_deficit}')

# Mark solutions
for t_sol in results:
    plt.plot([t_sol], [target_deficit], 'go', markersize=10)
    plt.annotate(f't = {t_sol:.2f}',
                xy=(t_sol, target_deficit),
                xytext=(t_sol, target_deficit - 0.15),
                ha='center', fontsize=12, fontweight='bold')
    plt.vlines(x=t_sol, ymin=0, ymax=target_deficit, color='g', linestyle=':', alpha=0.5)

# Plot settings
plt.title('Dissolved Oxygen Deficit - Streeter-Phelps Model', fontsize=16, fontweight='bold')
plt.xlabel('Time (days)', fontsize=14)
plt.ylabel('DO Deficit (mg/L)', fontsize=14)

```

```

plt.grid(True, alpha=0.3)
plt.legend(loc='best', fontsize=12)
plt.ylim(bottom=0)
plt.xlim(0, min(max(results)*2 if results else 10, 20))

# Add parameter information
param_text = (f'Parameters:\n'
              f'K1 = {desoxygenation_rate_constant} /day\n'
              f'K2 = {reaeration_rate_constant} /day\n'
              f'Lo = {bod_ultimate_mixing_zone} mg/L\n'
              f'Do = {initial_disolved_oxygen_deficit} mg/L')

plt.figtext(0.02, 0.02, param_text, fontsize=12,
           bbox=dict(facecolor='white', alpha=0.8, boxstyle='round,pad=0.5'))
y0, y1 = plt.ylim()      # get current (bottom, top)
plt.ylim(y1, y0)         # swap them
plt.tight_layout()
plt.show()

return results

# Example usage with the specified parameters
if __name__ == "__main__":

    # Target deficit to solve for
    target_deficit = 0.8 # Same as initial

    print(f"Solving for time when deficit = {target_deficit} mg/L:")
    result_times = solve_for_time(target_deficit, desoxygenation_rate_constant,
                                  reaeration_rate_constant, bod_ultimate_mixing_zone,
                                  initial_disolved_oxygen_deficit)

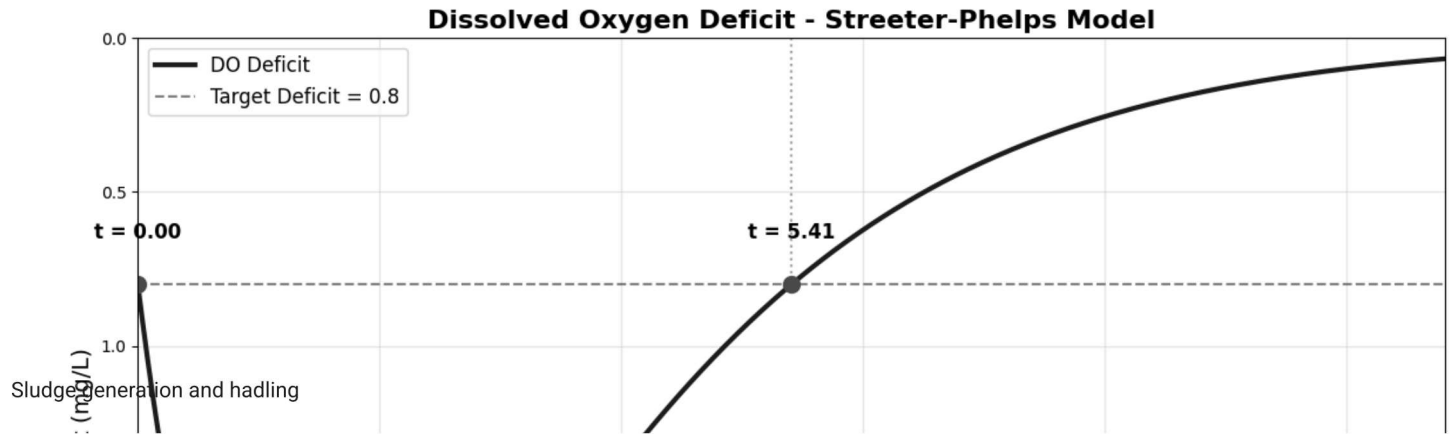
    # Print formatted results
    if result_times:
        if len(result_times) == 1:
            print(f"\nTime at which DO deficit reaches {target_deficit} mg/L: {result_times[0]:.6f} days")
        else:
            print(f"\nDO deficit reaches {target_deficit} mg/L at multiple times:")
            for i, t in enumerate(result_times):
                print(f"  Solution {i+1}: t = {t:.6f} days")

            lenght_same_DO_level=round(result_times[1]*86400*stream_velocity,2) #m, this is a value taken from the briefing.
            print('Length when it is reached the same DO level: ' +str(lenght_same_DO_level)+' m')

    else:
        print(f"\nNo solution found for target deficit = {target_deficit} mg/L")

```


→ Solving for time when deficit = 0.8 mg/L:
 Maximum deficit = 2.5319 mg/L at $t = 1.4414$ days
 Initial deficit already equals target (0.8)
 Deficit reaches 0.8 at $t = 5.4099$ days (falling limb)



```
#BOD_required_to_effluent g/m3, Xe
SRT=sludge_age #days, at Q average
tss_vss_primary_effluent_percentage=85 %%volatile, We suppose this value since we have a domestic and comercial wastewater.
#Therefore, it is not expected a large amount of non-biodegradable solids.
vss_primary_effluent_percentage=30 %% non-biodegradable. Same reason as above.
primary_sludge_solids_content= 7 %% This value is taken from table 18-8 Metcalf & Eddy. It ranges from 2-8% for primary sludge.
secondary_sludge_solids_content= 2 %% This value is taken from table 18-9 Metcalf & Eddy. It ranges from 0.4-2.5% for primary sludge.
specific_gravity_both_sludges=1.03 #dimensionless, this is the specific gravity of both sludges. Table 5.21 Metcalf & Eddy.
specific_weight_water=1000 #kg/m3\n",

peak_flow_rate=Q_new_peak #m3/day
average_flow_rate=Q_total_projected_average #m3/day

tss_peak_raw=tss_concentration #mg/l\n",
tss_average_raw=tss_concentration #mg/l\n",

tss_average_effluent=tss_concentration-0.5114*tss_concentration #mg/l #This value is taken from primary clarifier design.
tss_peak_effluent=tss_concentration-0.5882*tss_concentration #mg/l #This value is taken from primary clarifier design.

#Primary sludge volume

primary_sludge_production_average=round(average_flow_rate*(tss_average_raw-tss_average_effluent)/1000,2)
print ('Average flow primary sludge in kg is: '+str(primary_sludge_production_average))
```